FM School

Project guide for the SEPM lab

Run the application, follow the code, and explain the tests in the viva. GitHub repository

1. Set up and run
2. Understand the code
3. Demo the main workflows
4. Use the JSON API
5. Know the nine new tests
6. Run CI and make changes
7. Prepare for the viva
8. Application code walkthrough
9. Six unit tests in code
10. Three integration tests in code

1 Set up and run

FM School manages Teacher and Student accounts, profiles, course requests, and marks. The Admin manages records. Teachers and Students work with their own profiles and enrollments. The project uses Java 25, Spring Boot, Maven, and PostgreSQL 18.6.

1.1 Start on a Windows lab PC

Install Git, JDK 25, Docker Desktop, and IntelliJ. Start Docker Desktop and wait for its engine. Select JDK 25 for the IntelliJ project and Maven importer. For terminal commands, set `JAVA_HOME` to the JDK directory and check `java -version` and `.\mvnw.cmd -version`.

```
git clone https://github.com/FahimHossen121/fm-school-academic.git
cd fm-school-academic
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\run-demo.ps1
```

The runner starts PostgreSQL and Spring Boot, applies the demo seed after startup, and opens localhost:8080/login. The seed resets the local Admin password to the demo value. Use it on a lab database.

Role	Username / password	First page
Admin	`admin` / `12345678`	Admin home
Teacher	`teacher2` / `teacher2`	My Profile
Student	`student2` / `student2`	My Profile

1.2 Run the steps manually if needed

Start the database with `docker compose up -d postgres`. In IntelliJ, run `FmSchoolApplication` and wait for Liquibase to finish. Load the demo rows from a second terminal and refresh the login page.

```
Get-Content .\demo\seed-demo-data.sql |
  docker compose exec -T postgres psql -v ON_ERROR_STOP=1 -U myuser -d fm_school_db
```

Stop Spring Boot with the IDE Stop button if you started it there. If you used the runner, identify its Java process before stopping it. `docker compose stop` stops PostgreSQL and keeps the volume. GitHub stores code and migrations. Use the seed to recreate demo data on another PC.

2 Understand the code

A web request enters Spring Security, which checks the user's identity and route permissions. The controller reads the request. The service applies business rules. The repository reads or writes the database. The controller returns an HTML view or JSON.



2.1 Find the right file

Java classes use the package `com.example.fmschool`. In IntelliJ, open files by name with Ctrl+Shift+N.

File or package	Responsibility
`SecurityConfig.java`	Form login, API authentication, role rules, and redirects.
`StudentController.java`	Admin Student CRUD endpoints and HTTP responses.
`EnrollmentApiController.java`	API requests to apply, invite, respond, remove, and edit.
`EnrollmentService.java`	Recipient checks, one accepted course, and Teacher edit authority.
`ProfileService.java`	Own profile, Admin edits, and account/profile deletion.
`PeopleApiService.java`	Admin account/profile changes and JSON response DTOs.
`entity/` and `repository/`	Java representations of stored data and the interfaces that query them.
`dto/`	Data shapes for requests and responses. Validation checks incoming fields.
`templates/`	Thymeleaf HTML views under `src/main/resources/`.

2.2 The four main tables

- **Accounts** hold the username, role, and password hash.
- **Students** hold a name and roll number. **Teachers** hold a name and one course. Their account links connect profiles to login identities.
- **Enrollments** connect a Student and Teacher. They record who requested the link, its status, and optional marks. One Student can have several pending applications and at most one accepted course.

Liquibase creates and updates the schema. Its changelog defines the one-accepted-course constraint and the foreign-key rules for enrollment deletion. Hibernate validates the schema and maps entities to tables.

2.3 Follow a profile update

When a Student submits My Profile, the controller validates the form and calls `ProfileService`. The service looks up the signed-in account, then finds and saves that account's Student profile. The name and roll belong to the profile. The username and password hash belong to the account.

3 Demo the main workflows

Role	Typical actions
Admin	Manage Teacher and Student accounts. Directly associate a Student with a Teacher and manage requests.
Teacher	Edit own profile, invite Students, respond to applications, and edit names, rolls, and marks for accepted Students.
Student	Edit own profile, apply to Teachers, respond to invitations, and view own marks.

3.1 A quick lab demonstration

1. Sign in as `teacher2`. Find an accepted Student in the enrollment board, open the edit page, and enter a mark such as 85.50.
2. Sign out and sign in as that Student. Check the mark on the enrollment board. Open My Profile to show how the signed-in identity selects the record.
3. To show a pending request, use a new Student and Teacher. Complete both profiles, then apply as the Student. Sign in as the Teacher and accept. You can also send an invitation as the Teacher and respond as the Student.
4. Sign in as Admin and open Manage Teachers or Manage Students. Show the management actions across accounts and profiles.

The seed already enrolls the demo Students. Use a fresh Student account when demonstrating an application or invitation.

3.2 Rules to check while demonstrating

- The recipient or Admin can accept or reject a pending request. A Teacher gains edit permission when their enrollment with the Student is accepted.
- A Student must leave the accepted course before accepting another. Removing an association leaves both accounts usable.
- Marks range from 0 to 100 with up to two decimal places. An empty mark means ungraded. A mark of zero is a grade.
- A Teacher can edit an accepted Student only through their own enrollment. Use another Teacher account to demonstrate the denial.
- Deleting a Teacher account removes the profile and its enrollment links. The affected Student accounts survive. Use disposable accounts for the demo.

3.3 Troubleshoot the demo

Symptom	First check
Database connection error	Run `docker compose ps`. Check Docker Desktop and the port mapping in `compose.yaml`.
Wrong Java version	Check terminal `java -version` and IntelliJ's Maven JDK.
Demo password rejected	Confirm the app started, then run the demo seed against the local lab database.
Port 8080 in use	Identify the process using the port. The runner and IntelliJ can start separate Java processes.
Styles look old	Rebuild and refresh. Rebuild Tailwind when changing its input or template classes.

4 Use the JSON API

The HTML controllers render Thymeleaf pages. The API controllers return JSON using the same services and ownership rules. A DTO carries selected fields between the app and the client. Response DTOs leave password hashes out of JSON.

4.1 Authentication and writes

Use your browser session or HTTP Basic authentication in Postman. For POST, PUT, and DELETE, first call `GET /api/csrf`. Keep the session cookie and send the returned token using the returned `headerName`. CSRF protection verifies that a browser write came from a client that obtained the token for that session.

Endpoint	Purpose
`GET /api/session`	See the authenticated username and roles.
`GET /api/csrf`	Obtain the token and header name.
`GET`, `PUT`, `DELETE /api/me`	View, edit, or remove your own Teacher or Student profile and login as applicable.
`/api/students` and `/api/teachers`	Admin lists and creates people. Add an ID to read, update, or delete one record.
`GET /api/enrollments`	Read the role-filtered board.
`POST /api/enrollments/applications`	Student applies with a `teacherId`.
`POST /api/enrollments/invitations`	Teacher invites with a `studentId`.
`POST /api/enrollments/associations`	Admin associates a Teacher and Student by profile IDs.
`POST /api/enrollments/{id}/accept`	Recipient or Admin accepts. Use `/reject` to decline.
`DELETE /api/enrollments/{id}`	Participant removes an accepted link. Admin can remove any link.
`PUT /api/enrollments/{id}/student`	Associated Teacher or Admin edits Student details and marks on an accepted link.

4.2 One request to show

As the associated Teacher, send this JSON to `PUT /api/enrollments/{id}/student`. The enrollment ID identifies the relationship. You can find it on the enrollment board.

```
{"name":"Student Two","roll":"9000002","marks":85.50}
```

A successful write returns 200. Repeat the call as an unrelated Teacher and expect 403. When accepting a second course, expect 409 and check that the original enrollment still exists.

Status codes. 200 success, 201 created, 204 success without a body, 400 invalid input, 401 authentication required, 403 permission or CSRF denied, 404 missing record, and 409 conflict with the current state.

5 Know the nine new tests

A unit test isolates one piece of logic. These six tests call `EnrollmentService` directly and use Mockito repositories. An integration test checks several parts working together. The three new integration tests use the full Spring application, MockMvc, Liquibase, and a separate PostgreSQL database. They read the stored results after HTTP requests complete.

5.1 Six unit tests

	Scenario	What it proves
U1	Accept twice	The retry returns 409. The record stays accepted and no second save runs.
U2	Reverse a rejected request	A Teacher invites a Student whose application they rejected. The same row reopens and the Student becomes the recipient.
U3	Filter the invitation list	Students who already have a course or lack a linked account stay out of the picker.
U4	Keep pending history	An enrolled Student still sees pending rows, while the board offers no new Teachers to apply to.
U5	Clear an existing mark	Clearing 78.25 saves a null mark, which means ungraded.
U6	Invite without a profile	A Teacher login without a profile gets a conflict and no enrollment is written.

5.2 Three integration tests

1. **Transfer courses.** Apply to two Teachers. Accept the first request, then attempt to accept the second. Check for 409 and confirm the second request stays pending in PostgreSQL. Remove the first enrollment, accept the second, and verify there is exactly one accepted row for the Student.
2. **Roll back a failed creation.** Ask Admin to create a Teacher with valid account details and no course. The profile step returns 400. Look up the username in PostgreSQL and verify the earlier account insert was rolled back.
3. **Delete a Teacher without losing a Student.** Create an accepted enrollment, delete the Teacher as Admin, and query the database. The Teacher account, profile, and enrollment are gone. The Student profile survives and the Student can still sign in with the original credentials.

The existing application startup check also runs in the integration phase.

5.3 Explain the rollback test in the viva

The HTTP response tells the client the course is required. Querying the accounts table shows that the earlier insert was rolled back. A response assertion alone could pass even if an unused account were left in the database.

6 Run CI and make changes

6.1 Run tests on your PC

Run unit and isolated web tests with `.\mvnw.cmd test`. To run the integration tests too, start a disposable PostgreSQL container on port 55432. The demo database uses port 5432, so both can run independently.

```
docker run -d --rm --name fm-school-lab-test `
  -p 127.0.0.1:55432:5432 `
  -e POSTGRES_DB=fm_school_test -e POSTGRES_USER=fm_school_test `
  -e POSTGRES_PASSWORD=fm_school_test postgres:18.6
docker exec fm-school-lab-test pg_isready -U fm_school_test -d fm_school_test
```

Once `pg_isready` reports that the database accepts connections, run the suite and stop the disposable container when finished.

```
.\mvnw.cmd clean verify
docker stop fm-school-lab-test
```

The integration profile points to this dedicated database. Maven Surefire runs `*Tests`. Maven Failsafe runs `*IT` during `verify`. Reports are in `target/surefire-reports` and `target/failsafe-reports`.

6.2 What GitHub Actions does

The workflow is `.github/workflows/tests.yml`. A push or pull request starts an Ubuntu job. GitHub starts a clean PostgreSQL service, waits for its health check, sets up Java 25, and executes `mvn verify`. Open the Actions tab, select FM School tests, then open the `verify` job to see the test totals and result.

6.3 Make a small requested change

Trace the feature from controller to service before editing. For a marks range change, check the form, input validation, database constraint, and boundary tests. Add a new Liquibase changeset when the stored schema changes.

Change	Start here
Page text or layout	Thymeleaf files in `src/main/resources/templates/`.
Colors and spacing	Tailwind source in `src/main/resources/styles/`. Run `npm.cmd ci`, then `npm.cmd run css:build`. Commit generated CSS with the source change.
Permission or workflow	`SecurityConfig` for route rules. The relevant service for record ownership and business rules.
Database column	Entity and a new Liquibase changeset. Update DTOs, forms, and tests as needed.

Pull changes before working on a different PC. After editing, run the tests and review `git diff`. Commit the specific files, push, and check the new Actions run.

7 Prepare for the viva

7.1 How did you create the REST API?

Open `StudentController.java`. Show `@RestController`, `@RequestMapping("/api/students")`, and `@GetMapping`. The GET method delegates to `PeopleApiService.students()`, which queries the repository and maps results to response DTOs. In the POST method, `@RequestBody` binds JSON, `@Valid` checks the request, and creation returns 201 with a Location header.

7.2 How did you use Docker?

Open `compose.yaml`. PostgreSQL 18.6 runs in Docker, and Spring Boot runs on the host through IntelliJ or Maven. Compose sets the database name and login, publishes 5432 to localhost, and attaches `postgres_data` to preserve data across ordinary container recreation. Show `docker compose ps`. Explain that the PostgreSQL driver connects Java to the database.

7.3 How did you implement Spring Data JPA?

Show `Student.java`, `StudentRepository.java`, then `StudentService.updateStudent()`. The entity maps Java fields to columns. The repository provides methods such as `findById` and `save`. The service loads a Student, changes its fields, and saves within a transaction. Hibernate handles the mapping. Liquibase manages changes to the schema.

7.4 How did you implement Spring Security?

Open `SecurityConfig.java`. Show the encoder, account lookup through `UserDetailsService`, and the web and API filter chains. A user first proves who they are. Route rules decide what their role may access. Open `EnrollmentService.requireEditor()` for the record-level check. A Teacher can edit the Student when the accepted enrollment belongs to that Teacher's account. The server applies this rule to direct API requests too.

7.5 What distinguishes the two test types?

In `EnrollmentServiceUnitTests.java`, Mockito supplies the repository responses and the test checks the service logic. In `EnrollmentPersistenceIT.java`, a request passes through security, controller, service, and real database transactions. Show how the rollback test checks both the response and the absence of the account in PostgreSQL. Finish by showing the Actions run.

7.6 Original material

- Full project guide and detailed viva notes
- Windows lab setup and feature walkthroughs
- JSON API reference and Tailwind workflow
- REST API, Spring Boot, and Docker lab report

8 Application code walkthrough

The excerpts keep the repository's line numbers. Click the source label to open the exact revision. Start with the POST handler below, then follow the service, database, and security code as needed.

8.1 Read an HTTP request and create a Student

A request to `POST /api/students` carries login details and a nested Student profile. The controller turns that request into a call to the application service.

`StudentController.java`, lines 31–35

```
31 @PostMapping
32 public ResponseEntity<PeopleApiService.StudentView> create(@Valid @RequestBody CreatePersonRequest
       request) {
33     var created = people.createStudent(request);
34     return ResponseEntity.created(URI.create("/api/students/" + created.id())).body(created);
35 }
```

1. `@PostMapping` combines with the class path `/api/students`. The request method and path together select this handler.
2. `@RequestBody` binds JSON into `CreatePersonRequest`. `@Valid` runs field checks before the method executes. The DTO's nested `@Valid` also checks the profile.
3. The controller delegates to `people.createStudent(request)`. The service creates the account and profile, then builds a response view.
4. `ResponseEntity.created` returns 201. Its Location header points to the new Student profile by ID. The response body is the view object.

`CreatePersonRequest.java`, lines 6–11

```
 6 public record CreatePersonRequest(
 7         @NotBlank @Size(min = 3, max = 50)
 8         @Pattern(regexp = "[a-zA-Z0-9._-]+") String username,
 9         @NotBlank @Size(min = 8, max = 64) String password,
10         @NotNull @Valid ProfileForm profile) {
11 }
```

`PeopleApiService.java`, lines 48–52

```
48 public StudentView createStudent(CreatePersonRequest request) {
49     Long accountId = createIdentity(request, Role.STUDENT);
50     profiles.updateManagedProfile(accountId, request.profile());
51     return studentView(students.findByAccountId(accountId).orElseThrow());
52 }
```

Which ID is which? The account ID identifies the login record. The Student profile ID identifies the row addressed by `/api/students/{id}`. The service looks up the profile using the account ID it just created, then returns the profile view.

Follow the GET path. The class's list handler calls `PeopleApiService.students()`. The service reads entities with `findAll()` and maps each entity to `StudentView`. A view makes the JSON fields explicit, which keeps the password hash out of the response.

8.2 Map the Student profile and save an update

An entity represents a database record. The repository provides operations on that entity. The service decides which record to load and which fields to update.

`Student.java`, lines 7–27

```java
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Getter
    @Setter
    @com.fasterxml.jackson.annotation.JsonIgnore
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "account_id", unique = true)
    private Account account;

    @Setter
    @Getter
    private String name;
    @Setter
    @Getter
    private String roll;
```

`StudentRepository.java`, lines 6–13

```java
public interface StudentRepository extends JpaRepository<Student, Long> {
    java.util.Optional<Student> findByAccountId(Long accountId);

    @org.springframework.data.jpa.repository.Lock(jakarta.persistence.LockModeType.
    PESSIMISTIC_WRITE)
    @org.springframework.data.jpa.repository.Query("select s from Student s where s.id = :id")
    java.util.Optional<Student> findLockedById(
            @org.springframework.data.repository.query.Param("id") Long id);
}
```

`StudentService.java`, lines 43–49

```java
@Transactional
public void updateStudent(Long id, StudentForm form) {
    Student student = getStudentById(id);
    student.setName(form.getName());
    student.setRoll(form.getRoll());
    studentRepository.save(student);
}
```

- `@Id` marks the profile's primary key. `IDENTITY` lets PostgreSQL assign the ID. The unique `account_id` links at most one Student profile to the same account. Lazy loading defers fetching that account until needed.
- `JpaRepository<Student, Long>` declares the entity and ID type. Spring Data provides `findById` and `save`. It derives `findByAccountId` from the relationship and property names.
- `findLockedById` uses JPQL, which names the Java entity and its fields. Its pessimistic write lock holds the row until the transaction ends. That matters for two actions that could try to enroll the same Student.
- In `updateStudent`, a lookup identifies the record, setters change the in-memory entity, and the save is part of a transaction. The transaction commits those changes to the database.

8.3 Sign in and choose a landing page

Authentication answers who the user is. It loads an account by username and checks the supplied password against the stored hash. The success handler then chooses the landing page.

`SecurityConfig.java`, lines 22–49

```
22 @Bean
23 public PasswordEncoder passwordEncoder() {
24     return PasswordEncoderFactories.createDelegatingPasswordEncoder();
25 }
26
27 @Bean
28 public UserDetailsService userDetailsService(AccountRepository accounts) {
29     return username -> {
30         var account = accounts.findByUsername(username)
31                 .orElseThrow(() -> new UsernameNotFoundException("Account not found"));
32         return User.withUsername(account.getUsername())
33                 .password(account.getPasswordHash())
34                 .roles(account.getRole().name())
35                 .build();
36     };
37 }
38
39 @Bean
40 public AuthenticationSuccessHandler roleLoginSuccessHandler() {
41     return (request, response, authentication) -> {
42         boolean admin = authentication.getAuthorities().stream()
43                 .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
44         var handler = new SavedRequestAwareAuthenticationSuccessHandler();
45         handler.setAlwaysUseDefaultTargetUrl(true);
46         handler.setDefaultTargetUrl(admin ? "/" : "/me");
47         handler.onAuthenticationSuccess(request, response, authentication);
48     };
49 }
```

1. The encoder supports stored password hashes with an algorithm identifier. Spring Security uses it to compare passwords during login.
2. `UserDetailsService` loads the username, password hash, and role. A missing account raises `UsernameNotFoundException`.
3. `roles(...)` gives Spring Security an authority such as `ROLE_TEACHER`. Rules written as `hasRole("TEACHER")` check that authority.
4. The success handler checks for `ROLE_ADMIN`. It redirects Admin to `/` and Teacher or Student to `/me`. The destination comes from the role Spring loaded from the account.

Where the tests touch it. Most MockMvc requests set a test identity directly, so they exercise authorization. The Teacher-deletion integration test goes further by submitting the Student's real credentials through the form-login filter after deletion.

8.4 Choose the security rules for a request

Spring Security uses the first matching filter chain. The API chain has order 1. The general web chain has order 2.

`SecurityConfig.java`, lines 51–86

```
51 @Bean
52 @Order(2)
53 public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
54     return http
55             .authorizeHttpRequests(auth -> auth
56                     .dispatcherTypeMatchers(DispatcherType.ERROR, DispatcherType.FORWARD).
       permitAll()
57                     .requestMatchers("/login", "/signup", "/signup/student", "/signup/teacher",
58                             "/css/**", "/favicon.ico").permitAll()
59                     .requestMatchers("/students", "/students/**",
60                             "/api/students", "/api/students/**",
61                             "/admin", "/admin/**").hasRole("ADMIN")
62                     .anyRequest().authenticated())
63             .formLogin(form -> form.loginPage("/login")
64                     .successHandler(roleLoginSuccessHandler()).permitAll())
65             .httpBasic(Customizer.withDefaults())
66             .logout(logout -> logout.logoutSuccessUrl("/login?logout"))
67             .build();
68 }
69
70 @Bean
71 @Order(1)
72 public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
73     return http.securityMatcher("/api/**")
74             .authorizeHttpRequests(auth -> auth
75                     .requestMatchers("/api/students", "/api/students/**",
76                             "/api/teachers", "/api/teachers/**").hasRole("ADMIN")
77                     .anyRequest().authenticated())
78             .httpBasic(basic -> basic.authenticationEntryPoint((request, response, exception) ->
79                     apiError(response, 401, "Authentication required")))
80             .exceptionHandling(errors -> errors
81                     .authenticationEntryPoint((request, response, exception) ->
82                             apiError(response, 401, "Authentication required"))
83                     .accessDeniedHandler((request, response, exception) ->
84                             apiError(response, 403, "Access denied or missing CSRF token")))
85             .build();
86 }
```

- `securityMatcher("/api/**")` selects the API chain. The route rules inside it restrict the people collections to Admin and require authentication for the other API routes. Basic authentication is available for API clients.
- The web chain opens login, signup, and static CSS. It protects Admin routes and configures the form login and logout pages.
- `401` means authentication is required. `403` can mean a role or ownership check failed, or a write lacked a valid CSRF token.

A route can permit a Teacher to call an enrollment endpoint. The service must still check whether the specific enrollment belongs to that Teacher. The next two sections show that distinction in code.

8.5 Apply, invite, and respond to an enrollment

A Student application and a Teacher invitation use the same enrollment table. `requestedBy` records which side sent the request, so the service can identify the recipient. A rejected pair can reopen through the shared `request` method.

`EnrollmentService.java`, lines 105–121

```
105 @Transactional
106 public void respond(Long id, boolean accept) {
107     Account actor = currentAccount();
108     Enrollment enrollment = enrollment(id);
109     // Check who may act before exposing the current request state.
110     if (actor.getRole() != Role.ADMIN && !isRecipient(enrollment, actor)) {
111         throw forbidden();
112     }
113     if (enrollment.getStatus() != EnrollmentStatus.PENDING) {
114         throw conflict("Only pending requests can be accepted or rejected");
115     }
116     if (accept) {
117         requireAvailable(enrollment.getStudent().getId(), enrollment.getTeacher().getId());
118     }
119     enrollment.setStatus(accept ? EnrollmentStatus.ACCEPTED : EnrollmentStatus.REJECTED);
120     enrollments.save(enrollment);
121 }
```

`EnrollmentService.java`, lines 136–153

```
136 private void request(Teacher teacher, Student student, Role sender) {
137     student = lockStudent(student.getId());
138     requireAvailable(student.getId(), teacher.getId());
139     Enrollment enrollment = enrollments.findLockedPair(teacher.getId(), student.getId())
140             .orElseGet(Enrollment::new);
141     if (enrollment.getStatus() == EnrollmentStatus.ACCEPTED) {
142         throw conflict("This Student is already associated with this Teacher");
143     }
144     if (enrollment.getStatus() == EnrollmentStatus.PENDING) {
145         throw conflict("A request is already pending; its recipient must respond");
146     }
147     // A rejected pair can apply/invite again, without duplicating the association.
148     enrollment.setTeacher(teacher);
149     enrollment.setStudent(student);
150     enrollment.setRequestedBy(sender);
151     enrollment.setStatus(EnrollmentStatus.PENDING);
152     enrollments.save(enrollment);
153 }
```

- `respond` checks the actor before checking the request status. Only Admin or the recipient can respond. A second acceptance encounters the status guard and returns a conflict before the save.
- Accepting also calls `requireAvailable`. It checks whether the Student already has an accepted enrollment with another Teacher.
- `request` locks the Student and finds the existing Teacher–Student pair. For a rejected request it reuses the row, sets the new sender, and returns the status to pending. A pending pair already has a recipient who needs to respond.

Concurrency. `enrollment(id)` locks the Student before the enrollment. That serializes competing acceptances for the same Student. The database also enforces the one-accepted-course rule through a partial unique index on `student_id` for accepted rows. The course transfer test exercises the rule sequentially.

8.6 Edit marks through the accepted enrollment

Teacher edit access depends on the role, the enrollment's status, and the account that owns the Teacher profile. The service uses the enrollment ID to load that relationship before writing either record.

`EnrollmentService.java`, lines 204–225

```
204 @Transactional
205 @PreAuthorize("hasAnyRole('TEACHER', 'ADMIN')")
206 public void updateEnrolledStudent(Long id, @Valid EnrolledStudentForm form) {
207     Enrollment enrollment = enrollment(id);
208     requireEditor(enrollment, currentAccount());
209     Student student = enrollment.getStudent();
210     student.setName(form.getName());
211     student.setRoll(form.getRoll());
212     enrollment.setMarks(form.getMarks());
213     students.save(student);
214     enrollments.save(enrollment);
215     // Account username/password/role are deliberately untouched.
216 }
217
218 private void requireEditor(Enrollment enrollment, Account actor) {
219     if (enrollment.getStatus() != EnrollmentStatus.ACCEPTED
220             || (actor.getRole() != Role.ADMIN
221                 && !(actor.getRole() == Role.TEACHER
222                     && Objects.equals(enrollment.getTeacher().getAccount().getId(), actor.getId())
    ))) {
223         throw forbidden();
224     }
225 }
```

`db.changelog-master.yaml`, lines 170–173

```
170     sql: CREATE UNIQUE INDEX enrollments_one_accepted_course ON enrollments (student_id) WHERE
        status = 'ACCEPTED'
171 - sql:
172     dbms: postgresql
173     sql: ALTER TABLE enrollments ADD CONSTRAINT enrollments_marks_range CHECK (marks IS NULL OR (
        marks >= 0 AND marks <= 100))
```

1. `@PreAuthorize` permits Teacher and Admin roles through the method boundary. `requireEditor` requires an accepted enrollment. For Teachers it compares the enrollment's Teacher account ID with the actor's account ID.
2. The name and roll belong to the Student profile. The mark belongs to the enrollment, so both are updated within the transaction. Login details stay in the linked account record.
3. `form.getMarks()` can return null. The service stores that null as ungraded. Zero remains a valid numeric mark. The schema permits null or a value between 0 and 100.

What the board shows. `getBoard()` gets enrollment rows separately from the lists of people the user can invite or apply to. That distinction lets a Student see pending history even when an accepted enrollment hides the new-application picker. A Teacher's picker filters out Students who already have an accepted course or have no account.

8.7 Roll back creation and clean up deletions

The Admin API works across two kinds of records. Account creation writes the identity first and profile creation fills in the role-specific record afterward.

PeopleApiService.java, lines 13–16

```
13 @Service
14 @PreAuthorize("hasRole('ADMIN')")
15 @Transactional
16 public class PeopleApiService {
```

PeopleApiService.java, lines 78–82

```
78 public TeacherView createTeacher(CreatePersonRequest request) {
79     Long accountId = createIdentity(request, Role.TEACHER);
80     profiles.updateManagedProfile(accountId, request.profile());
81     return teacherView(teachers.findByAccountId(accountId).orElseThrow());
82 }
```

ProfileService.java, lines 142–150

```
142 } else if (account.getRole() == Role.TEACHER) {
143     if (form.getCourse() == null || form.getCourse().isBlank()) {
144         throw new IllegalArgumentException("Course is required");
145     }
146     Teacher teacher = teachers.findByAccountId(account.getId()).orElseGet(Teacher::new);
147     teacher.setAccount(account);
148     teacher.setName(form.getName());
149     teacher.setCourse(form.getCourse());
150     teachers.save(teacher);
```

ProfileService.java, lines 156–163

```
156 private void delete(Account account) {
157     students.findByAccountId(account.getId()).ifPresent(students::delete);
158     teachers.findByAccountId(account.getId()).ifPresent(teachers::delete);
159     // Remove dependent records before the account, preserving unrelated people.
160     students.flush();
161     teachers.flush();
162     accounts.delete(account);
163 }
```

- The class-level `@Transactional` on `PeopleApiService` gives `createTeacher` a transaction that covers both steps. The nested account and profile service calls join that transaction.
- A missing course reaches the Teacher branch of `save` and throws an `IllegalArgumentException`. The API handler returns 400. The transaction rolls back, so the account written by the first step disappears.
- Deleting an account removes its profiles first and flushes those deletions. The enrollment foreign keys have `ON DELETE CASCADE`, so the linked enrollment rows disappear when the Teacher profile is deleted.
- The Student profile belongs to another account. The cascade follows the deleted Teacher's enrollment links and leaves the Student's login usable.

What to show in the lab. Open integration test I2 for the rollback query and I3 for the row checks plus the Student's login after the Teacher is gone.

8.8 Connect the web form and the database

Thymeleaf renders the profile form with values supplied by Spring MVC. The browser submits the form to the route shown in the generated HTML.

`profile.html`, lines 27–51

```
27 <h2>Personal information</h2>
28 <form
29   th:action="@{${savePath}}"
30   th:object="${profileForm}"
31   method="post"
32   class="field-form"
33 >
34   <ul role="alert" th:if="${#fields.hasAnyErrors()}">
35     <li
36       th:each="error : ${#fields.allErrors()}"
37       th:text="${error}"
38     ></li>
39   </ul>
40   <p>
41     <label for="name">Name</label>
42     <input id="name" type="text" th:field="*{name}" />
43   </p>
44   <p th:if="${isStudent}">
45     <label for="roll">Roll</label>
46     <input
47       id="roll"
48       inputmode="numeric"
49       type="text"
50       th:field="*{roll}"
51     />
```

- `th:action` generates the route from the controller's save path. `th:object` selects the profile form object. `th:field` connects an input to one of its fields.
- The error list displays validation messages after invalid input. `isStudent` selects the Student roll input. The server also validates the profile before saving it.
- The shared layout loads the compiled Tailwind stylesheet from `/css/app.css`. Changing Tailwind source requires a CSS build.

`compose.yaml`, lines 1–14

```
1  services:
2    postgres:
3      image: 'postgres:18.6'
4      environment:
5        - 'POSTGRES_DB=fm_school_db'
6        - 'POSTGRES_PASSWORD=secret'
7        - 'POSTGRES_USER=myuser'
8      ports:
9        - '127.0.0.1:5432:5432'
10     volumes:
11       - 'postgres_data:/var/lib/postgresql'
12
13 volumes:
14   postgres_data:
```

The PostgreSQL service publishes its port to localhost. Spring Boot runs on the host and uses its PostgreSQL driver to reach that port. The named volume keeps database files across ordinary container recreation.

8.9 Read the GitHub Actions workflow

The workflow runs the same Maven verification goal used in local development. GitHub starts PostgreSQL in a service container on the Ubuntu runner.

`tests.yml`, lines 1–38

```yaml
name: FM School tests

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  verify:
    runs-on: ubuntu-latest
    timeout-minutes: 12
    services:
      postgres:
        image: postgres:18.6
        env:
          POSTGRES_DB: fm_school_test
          POSTGRES_USER: fm_school_test
          POSTGRES_PASSWORD: fm_school_test
        ports:
          - 55432:5432
        options: >-
          --health-cmd "pg_isready -U fm_school_test -d fm_school_test"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 12

    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '25'
          cache: maven
      - name: Run unit and PostgreSQL integration tests
        run: mvn --batch-mode --no-transfer-progress verify
```

1. `push` and `pull_request` schedule the checks. The workflow also supports a manual run.
2. The PostgreSQL service waits for `pg_isready`. Port 55432 on the runner maps to port 5432 inside the test database container.
3. Checkout retrieves the repository. Setup Java installs Temurin 25 and caches Maven dependencies.
4. `mvn verify` runs the isolated tests, packages the application, runs the PostgreSQL tests, and fails the job if either phase reports a failure.

The integration profile uses the same database name and port as this service. The workflow's successful `verify` job is the result to show during the demo.

9 Six unit tests in code

Each listing contains the complete test method. The fixture code below gives those methods a Student, a Teacher, and a pending application. The tests call `EnrollmentService` directly.

9.1 Set up the service and its inputs

`EnrollmentServiceUnitTests.java`, lines 33–37

```
33 private final EnrollmentRepository enrollments = mock(EnrollmentRepository.class);
34 private final AccountRepository accounts = mock(AccountRepository.class);
35 private final TeacherRepository teachers = mock(TeacherRepository.class);
36 private final StudentRepository students = mock(StudentRepository.class);
37 private final EnrollmentService service = new EnrollmentService(enrollments, accounts, teachers,
       students);
```

`EnrollmentServiceUnitTests.java`, lines 45–63

```
45 @BeforeEach
46 void setUp() {
47     teacherAccount = account(2L, "teacher1", Role.TEACHER);
48     studentAccount = account(3L, "student1", Role.STUDENT);
49     teacher = new Teacher();
50     teacher.setId(10L);
51     teacher.setAccount(teacherAccount);
52     student = student(20L, "Student One", studentAccount);
53     application = enrollment(7L, teacher, EnrollmentStatus.PENDING);
54
55     when(enrollments.findStudentId(7L)).thenReturn(Optional.of(20L));
56     when(students.findLockedById(20L)).thenReturn(Optional.of(student));
57     when(enrollments.findLockedById(7L)).thenReturn(Optional.of(application));
58 }
59
60 @AfterEach
61 void clearAuthentication() {
62     SecurityContextHolder.clearContext();
63 }
```

`EnrollmentServiceUnitTests.java`, lines 159–169

```
159 private void signInAs(Account account) {
160     when(accounts.findByUsername(account.getUsername())).thenReturn(Optional.of(account));
161     SecurityContextHolder.getContext().setAuthentication(new TestingAuthenticationToken(
162             account.getUsername(), null, "ROLE_" + account.getRole().name()));
163 }
164
165 private static ResponseStatusException assertStatus(HttpStatus status, Runnable action) {
166     var error = assertThrows(ResponseStatusException.class, action::run);
167     assertEquals(status, error.getStatusCode());
168     return error;
169 }
```

- The four repository mocks stand in for database calls. The service itself is the real implementation.
- `setUp()` runs before every test. Teacher account 2 owns Teacher profile 10. Student account 3 owns Student profile 20. Application 7 connects them and starts as a request from the Student.
- The three repository stubs let the service follow application 7 to Student 20 and load the same application object. Mockito supplies those values when the service calls the repository.
- `signInAs` sets the test identity and stubs account lookup. The cleanup method clears the security context after each test.

9.2 U1. An acceptance retry must leave the original result intact

`EnrollmentServiceUnitTests.java`, lines 65–77

```java
@Test
void retryingAcceptanceReturnsConflictWithoutSavingAgain() {
    signInAs(teacherAccount);

    service.respond(7L, true);
    assertEquals(EnrollmentStatus.ACCEPTED, application.getStatus());
    verify(enrollments).save(application);
    clearInvocations(enrollments);

    assertStatus(HttpStatus.CONFLICT, () -> service.respond(7L, true));
    assertEquals(EnrollmentStatus.ACCEPTED, application.getStatus());
    verify(enrollments, never()).save(any());
}
```

1. **Arrange.** Sign in as the Teacher who received the pending application. Use the shared application with ID 7.
2. **Act the first time.** `respond(7L, true)` accepts the application. Check that it is accepted and that the service called `save(application)`.
3. **Separate the two calls.** `clearInvocations` clears Mockito's recorded calls while keeping its stubs and the accepted application object. The next verification now measures only the retry.
4. **Act again and assert.** Expect 409. Check the same record still says accepted and verify that no repository save followed the retry.

Why it matters. A user can click twice or retry a request after a slow response. The service must recognize that the first acceptance already took effect. Checking the exception alone would miss a second write.

Find the production guard. In `EnrollmentService.respond`, the guard requires `PENDING`. The first call changes the status to `ACCEPTED`. The retry stops at the guard before reaching `enrollments.save`.

9.3 U2. A rejected application can reopen as an invitation

`EnrollmentServiceUnitTests.java`, lines 79–100

```java
@Test
void rejectedApplicationReopensAsInvitationWithStudentAsRecipient() {
    signInAs(teacherAccount);
    service.respond(7L, false);
    assertEquals(EnrollmentStatus.REJECTED, application.getStatus());

    when(teachers.findByAccountId(2L)).thenReturn(Optional.of(teacher));
    when(enrollments.findLockedPair(10L, 20L)).thenReturn(Optional.of(application));
    clearInvocations(enrollments);

    service.invite(20L);

    assertEquals(EnrollmentStatus.PENDING, application.getStatus());
    assertEquals(Role.TEACHER, application.getRequestedBy());
    verify(enrollments).save(same(application));
    assertStatus(HttpStatus.FORBIDDEN, () -> service.respond(7L, true));
    assertEquals(EnrollmentStatus.PENDING, application.getStatus());

    signInAs(studentAccount);
    service.respond(7L, true);
    assertEquals(EnrollmentStatus.ACCEPTED, application.getStatus());
}
```

Action	Stored status	Sender and next actor
Student applies	Pending	Student sends. Teacher may respond.
Teacher rejects	Rejected	A new request can reopen the same pair.
Teacher invites	Pending	Teacher sends. Student may respond.
Student accepts	Accepted	The association is now active.

- The Teacher first acts as recipient. After rejecting, the mock returns the same row for that Teacher–Student pair.
- `verify(...).save(same(application))` checks object identity. This catches replacing the rejected row with a different object.
- The invitation changes both the status and `requestedBy`. The Teacher's attempt to accept it now gets 403. The Student then accepts it.

Viva point. The owner of the right to respond changes when the sender changes. Merely checking for `PENDING` would miss that rule.

9.4 U3. Show only eligible Students in the Teacher's picker

EnrollmentServiceUnitTests.java, lines 102–114

```
102 @Test
103 void teacherPickerExcludesOccupiedStudentsAndProfilesWithoutAccounts() {
104     signInAs(teacherAccount);
105     when(teachers.findByAccountId(2L)).thenReturn(Optional.of(teacher));
106     Student occupied = student(21L, "Occupied", account(4L, "occupied", Role.STUDENT));
107     Student legacy = student(22L, "Legacy", null);
108     when(students.findAll()).thenReturn(List.of(student, occupied, legacy));
109     when(enrollments.findStudentIdsByStatus(EnrollmentStatus.ACCEPTED)).thenReturn(List.of(21L));
110
111     var board = service.getBoard();
112
113     assertEquals(List.of(new EnrollmentService.StudentOption(20L, "Student One")), board.students
        ());
114 }
```

The repository returns three Students. The test marks profile 21 as already enrolled and gives profile 22 no account. Comparing the entire returned list proves both exclusions and confirms that profile 20 remains selectable.

9.5 U4. Keep pending rows visible when a Student has a course

EnrollmentServiceUnitTests.java, lines 116–131

```
116 @Test
117 void enrolledStudentKeepsPendingRowsButCannotPickANewTeacher() {
118     signInAs(studentAccount);
119     when(students.findByAccountId(3L)).thenReturn(Optional.of(student));
120     Teacher otherTeacher = new Teacher();
121     otherTeacher.setId(11L);
122     Enrollment accepted = enrollment(8L, otherTeacher, EnrollmentStatus.ACCEPTED);
123     when(enrollments.findForAccount(3L)).thenReturn(List.of(application, accepted));
124     when(teachers.findAll()).thenReturn(List.of(teacher, otherTeacher));
125
126     var board = service.getBoard();
127
128     assertTrue(board.teachers().isEmpty());
129     assertEquals(List.of(7L, 8L), board.rows().stream().map(EnrollmentService.Row::id).toList());
130     assertEquals(EnrollmentStatus.PENDING, board.rows().getFirst().status());
131 }
```

The existing accepted row empties the Teacher picker. The two row IDs prove that the board still shows both relationships, and the status assertion confirms the older request stays pending.

The distinction. The picker answers whom the user can apply to now. The rows show existing requests and enrollments. Filtering the picker should leave that history visible.

9.6 U5. Clearing a mark returns the enrollment to ungraded

`EnrollmentServiceUnitTests.java`, lines 133–146

```
133 @Test
134 void clearingAnExistingMarkRestoresTheUngradedState() {
135     signInAs(teacherAccount);
136     application.setStatus(EnrollmentStatus.ACCEPTED);
137     application.setMarks(new BigDecimal("78.25"));
138     EnrolledStudentForm form = new EnrolledStudentForm();
139     form.setName(student.getName());
140     form.setRoll(student.getRoll());
141
142     service.updateEnrolledStudent(7L, form);
143
144     assertNull(application.getMarks());
145     verify(enrollments).save(application);
146 }
```

The accepted enrollment starts with 78.25. The form keeps the profile fields and leaves marks unset. The service sets the mark to null and saves the enrollment. This catches a change that would store zero or retain 78.25.

9.7 U6. A Teacher without a profile cannot invite

`EnrollmentServiceUnitTests.java`, lines 148–157

```
148 @Test
149 void teacherWithAnAccountButNoProfileCannotSendAnInvitation() {
150     signInAs(teacherAccount);
151     when(teachers.findByAccountId(2L)).thenReturn(Optional.empty());
152
153     var conflict = assertStatus(HttpStatus.CONFLICT, () -> service.invite(20L));
154
155     assertEquals("Complete My Profile before inviting", conflict.getReason());
156     verify(enrollments, never()).save(any());
157 }
```

The account exists and has the Teacher role. `findByAccountId(2L)` returns empty, so the service rejects the invite with the profile-completion message. `never().save(any())` verifies there was no enrollment write.

Why check the reason? Several enrollment rules can return 409. The reason proves that the missing profile caused the conflict.

9.8 Read the fixture helpers

These builders create plain entity objects. Fixed IDs make the scenarios readable. The integration tests use database-generated IDs instead.

EnrollmentServiceUnitTests.java, lines 171–196

```
171 private Enrollment enrollment(long id, Teacher assignedTeacher, EnrollmentStatus status) {
172     Enrollment enrollment = new Enrollment();
173     enrollment.setId(id);
174     enrollment.setTeacher(assignedTeacher);
175     enrollment.setStudent(student);
176     enrollment.setRequestedBy(Role.STUDENT);
177     enrollment.setStatus(status);
178     return enrollment;
179 }
180
181 private static Account account(long id, String username, Role role) {
182     Account account = new Account();
183     account.setId(id);
184     account.setUsername(username);
185     account.setRole(role);
186     return account;
187 }
188
189 private static Student student(long id, String name, Account account) {
190     Student student = new Student();
191     ReflectionTestUtils.setField(student, "id", id);
192     student.setName(name);
193     student.setRoll("00123");
194     student.setAccount(account);
195     return student;
196 }
```

- `enrollment` connects the supplied Teacher to the shared Student. Every fixture starts with the Student as sender unless the test changes it.
- `account` supplies the identity the service will resolve during the test. The Teacher profile's ID and the Teacher account's ID are separate values.
- `student` sets the generated ID through Spring's test utility because the production entity has no ID setter. Passing null for the account creates the legacy profile used by U3.

To run just these six tests from Windows PowerShell with JDK 25 configured:

```
.\mvnw.cmd '-Dtest=EnrollmentServiceUnitTests' test
```

10 Three integration tests in code

10.1 Start a real application context and use a test database

`EnrollmentPersistenceIT.java`, lines 24–35

```java
@SpringBootTest
@ActiveProfiles("integration")
class EnrollmentPersistenceIT {
    private static final String TEST_PASSWORD = "TestPass123!";
    @Autowired private WebApplicationContext context;
    @Autowired private JdbcTemplate db;
    private MockMvc mvc;

    @BeforeEach
    void setUp() {
        mvc = webAppContextSetup(context).apply(springSecurity()).build();
    }
```

`application-integration.properties`, lines 1–5

```
# Use a dedicated database for integration tests. Liquibase builds the schema.
spring.docker.compose.enabled=false
spring.datasource.url=jdbc:postgresql://127.0.0.1:55432/fm_school_test
spring.datasource.username=fm_school_test
spring.datasource.password=fm_school_test
```

1. `@SpringBootTest` loads the controllers, security, services, repositories, and Liquibase.
2. `@ActiveProfiles("integration")` selects a dedicated PostgreSQL database at port 55432. The profile disables the app's usual Compose startup. The test database is supplied by the local disposable container or GitHub Actions.
3. `webAppContextSetup` builds MockMvc from the real application context. `springSecurity()` attaches the filter chain. The request runs inside the test process.
4. `JdbcTemplate` checks stored rows after requests finish. These tests let service transactions commit or roll back normally so the query can check what actually persisted.

Identity and CSRF. `with(user(...).roles(...))` supplies the authenticated test identity. The service still resolves that username against a real account. `with(csrf())` supplies the token needed for a write. I3 also sends a login request with the stored Student's original password.

Where the files run. Maven Failsafe finds classes ending in `IT` during `verify`. The existing startup check and the three scenarios run in this phase.

10.2 Create fixtures through the API

`EnrollmentPersistenceIT.java`, lines 98–126

```
98  private Person createPerson(String collection, String name) throws Exception {
99      String username = testUsername();
100     String body = """
101             {"username":"%s","password":"%s","profile":{"name":"%s","roll":"00123","course":"SEPM
        "}}
102             """.formatted(username, TEST_PASSWORD, name);
103     mvc.perform(post("/api/" + collection).with(user("admin").roles("ADMIN")).with(csrf())
104                     .contentType("application/json").content(body))
105             .andExpect(status().isCreated());
106     String query = collection.equals("students")
107             ? "select s.id from students s join accounts a on s.account_id = a.id where a.username
         = ?"
108             : "select t.id from teachers t join accounts a on t.account_id = a.id where a.username
         = ?";
109     return new Person(db.queryForObject(query, Long.class, username), username);
110 }
111
112 private long apply(Person student, Person teacher) throws Exception {
113     mvc.perform(post("/api/enrollments/applications")
114                     .with(user(student.username()).roles("STUDENT")).with(csrf())
115                     .contentType("application/json")
116                     .content("{\"teacherId\":" + teacher.profileId() + "}"))
117             .andExpect(status().isCreated());
118     return db.queryForObject("select id from enrollments where student_id = ? and teacher_id = ?",
119             Long.class, student.profileId(), teacher.profileId());
120 }
121
122 private void accept(Person teacher, long id, int expectedStatus) throws Exception {
123     mvc.perform(post("/api/enrollments/{id}/accept", id)
124                     .with(user(teacher.username()).roles("TEACHER")).with(csrf()))
125             .andExpect(status().is(expectedStatus));
126 }
```

- `createPerson` posts to the Admin API and expects 201. The test looks up the new profile ID by joining it to the account's unique username.
- `apply` posts the Teacher profile ID as the Student. The following query retrieves the new enrollment ID for that pair.
- `accept` calls the enrollment route as the specified Teacher and checks the expected response. The same helper handles successful and conflicting attempts.

The usernames include UUIDs. This keeps fixture names distinct when tests run again against the same disposable database.

10.3 I1. A Student can transfer only after leaving the first course

`EnrollmentPersistenceIT.java`, lines 37–60

```java
@Test
void studentCanTransferOnlyAfterLeavingTheAcceptedCourse() throws Exception {
    var student = createPerson("students", "Student");
    var first = createPerson("teachers", "First Teacher");
    var second = createPerson("teachers", "Second Teacher");
    long firstRequest = apply(student, first);
    long secondRequest = apply(student, second);

    accept(first, firstRequest, 200);
    accept(second, secondRequest, 409);

    assertEquals("ACCEPTED", enrollmentStatus(firstRequest));
    assertEquals("PENDING", enrollmentStatus(secondRequest));

    mvc.perform(delete("/api/enrollments/{id}", firstRequest)
                    .with(user(student.username()).roles("STUDENT")).with(csrf()))
            .andExpect(status().isNoContent());
    accept(second, secondRequest, 200);

    assertEquals(0, count("select count(*) from enrollments where id = ?", firstRequest));
    assertEquals("ACCEPTED", enrollmentStatus(secondRequest));
    assertEquals(1, count("select count(*) from enrollments where student_id = ? and status = '
    ACCEPTED'",
            student.profileId()));
}
```

1. Create one Student and two Teachers. Submit two applications before accepting either one. This builds two real pending rows.
2. Accept the first application and expect 200. Accepting the second now returns 409. Queries confirm the first remains accepted and the second remains pending.
3. Remove the first enrollment as the Student. Expect 204. The second acceptance now succeeds because the first association is gone.
4. Check that the old row was deleted, the second row is accepted, and the Student has exactly one accepted enrollment.

What this proves. The rule is enforced across separate requests and committed database state. A failed acceptance leaves the pending row available after the Student leaves the original course.

10.4 I2. A failed profile save rolls back the account

`EnrollmentPersistenceIT.java`, lines 62–75

```
62 @Test
63 void teacherCreationRollsBackTheAccountWhenCourseIsMissing() throws Exception {
64     String username = testUsername();
65     String body = """
66             {"username":"%s","password":"%s","profile":{"name":"Test Teacher"}}
67             """.formatted(username, TEST_PASSWORD);
68
69     mvc.perform(post("/api/teachers").with(user("admin").roles("ADMIN")).with(csrf())
70                     .contentType("application/json").content(body))
71             .andExpect(status().isBadRequest())
72             .andExpect(jsonPath("$.message").value("Course is required"));
73
74     assertEquals(0, count("select count(*) from accounts where username = ?", username));
75 }
```

1. Build valid account data with a unique username and a valid profile name. Omit only the course. The request can reach the service's role-specific rule.
2. Post the request as Admin with a CSRF token. Check 400 and the message `Course is required`. That confirms the profile save rejected this request.
3. Query the accounts table by username. A count of zero proves the identity created earlier in the transaction was rolled back.

Walk through the call chain. `TeacherApiController` invokes `PeopleApiService.createTeacher`. The service creates the identity, then calls `updateManagedProfile`. The missing course throws an exception inside the transaction. PostgreSQL rolls back the account insert, and the exception handler returns 400.

Why query the database? A controller could return the expected status even if it left an account behind. The query detects that partial write.

10.5 I3. Delete a Teacher and keep the Student's login usable

`EnrollmentPersistenceIT.java`, lines 77–96

```
77 @Test
78 void deletingATeacherRemovesTheEnrollmentButStudentCanStillSignIn() throws Exception {
79     var student = createPerson("students", "Student");
80     var teacher = createPerson("teachers", "Teacher");
81     long request = apply(student, teacher);
82     accept(teacher, request, 200);
83     assertEquals("ACCEPTED", enrollmentStatus(request));
84
85     mvc.perform(delete("/api/teachers/{id}", teacher.profileId())
86                     .with(user("admin").roles("ADMIN")).with(csrf()))
87             .andExpect(status().isNoContent());
88
89     assertEquals(0, count("select count(*) from accounts where username = ?", teacher.username()))
       ;
90     assertEquals(0, count("select count(*) from teachers where id = ?", teacher.profileId()));
91     assertEquals(0, count("select count(*) from enrollments where id = ?", request));
92     assertEquals(1, count("select count(*) from students where id = ?", student.profileId()));
93     mvc.perform(formLogin("/login").user(student.username()).password(TEST_PASSWORD))
94             .andExpect(authenticated().withUsername(student.username()))
95             .andExpect(redirectedUrl("/me"));
96 }
```

1. Create a Teacher and Student, then accept the application. The initial database check confirms there is a real accepted enrollment to remove.
2. Delete the Teacher profile through the Admin API. Expect 204.
3. Check that the Teacher's account and profile are gone. Check that the enrollment is gone too. The Student profile must still exist.
4. Submit the Student's original credentials through form login. Check the authenticated username and the redirect to `/me`.

Why test login after deletion? The profile query proves the Student row survived. The login proves the linked account, password hash, and Student role still work through Spring Security. The test therefore covers both storage and a real user action.